



UNIVERSIDAD DE XALAPA

• ————— •
Saber / Trascender

José Alberto Manzanilla Alfaro

Ingeniería en IA

Programación Estructurada

Documento descriptivo y justificación pedagógica

Para esta entrega se propone agregar las funciones en el Agente de Discord organizadas por el tema de Programación Estructurada que cada una enseña. Se incluyen tanto las funciones pedagógicas que el usuario final percibe (validar variables, explicar operadores, generar cuestionarios, etc.) como las funciones de arquitectura que permiten que el bot se comporte como un Agente y no como un chatbot lineal (clasificación de intención, registro en el catálogo de herramientas y memoria de estado).

Sintaxis básica

- **Función:** `mostrar_palabras_reservadas()` -> str
 - **Propósito Pedagógico:** Enseña qué son las palabras reservadas en Python y por qué no se pueden usar como nombres de variable, con ejemplos comunes (if, else, for, while, def, class, return, True, False).
 - **Justificación:** No recibe argumentos. Retorna un str con la lista de palabras reservadas obtenida mediante el módulo keyword de Python y una breve explicación de su uso.
-
- **Función:** `mostrar_identificadores()` -> str
 - **Propósito Pedagógico:** Enseña las reglas de nombrado de identificadores en Python (empezar con letra o guion bajo, no usar palabras reservadas, no usar espacios ni caracteres especiales).
 - **Justificación:** No recibe argumentos. Retorna un str con las reglas básicas y ejemplos de identificadores válidos e inválidos.
-
- **Función:** `validar_variable(nombre: str)` -> str
 - **Propósito Pedagógico:** Enseña a aplicar las reglas de nombrado vistas en `mostrar_identificadores` sobre un caso concreto que el usuario propone, para que practique en lugar de solo leer la teoría.
 - **Justificación:** Recibe como argumento el texto enviado tras el comando (por ejemplo `!validar 123dato`). Retorna un str indicando si el nombre es válido o, en caso de no serlo, qué regla específica está incumpliendo.

- **Función:** `mostrar_tipos_datos_simples()` -> str
- **Propósito Pedagógico:** Enseña los tipos de datos simples de Python (int, float, str, bool) y un ejemplo de cada uno, como base antes de avanzar a tipos compuestos.
- **Justificación:** No recibe argumentos. Retorna un str con cada tipo de dato simple y un ejemplo ilustrativo.

- **Función:** `mostrar_tipos_datos_compuestos()` -> str
- **Propósito Pedagógico:** Enseña los tipos de datos compuestos de Python (listas, tuplas, diccionarios) y en qué se diferencian entre sí, anticipando el bloque de Estructuras de Datos.
- **Justificación:** No recibe argumentos. Retorna un str con cada tipo de dato compuesto y un ejemplo breve de declaración.

- **Función:** `mostrar_variables_constantes()` -> str
- **Propósito Pedagógico:** Enseña la diferencia entre una variable (su valor puede cambiar) y una constante (su valor no debería cambiar durante la ejecución), y la convención de Python de escribir constantes en mayúsculas.
- **Justificación:** No recibe argumentos. Retorna un str explicando la diferencia conceptual y un ejemplo de cada caso.

- **Función:** `mostrar_operadores_aritmeticos()` -> str
- **Propósito Pedagógico:** Enseña los operadores aritméticos de Python (suma, resta, multiplicación, división, módulo, potencia, división entera) mostrando un ejemplo numérico de cada uno.
- **Justificación:** No recibe argumentos. Retorna un str con cada operador, su símbolo y un ejemplo de uso con resultado.

- **Función:** `mostrar_operadores_logicos()` -> str

- **Propósito Pedagógico:** Enseña los operadores lógicos de Python (and, or, not) y cómo se combinan condiciones booleanas, como preparación para el bloque de Estructuras de Control.
- **Justificación:** No recibe argumentos. Retorna un str con cada operador lógico, su comportamiento y un ejemplo con su resultado (verdadero o falso).

Estructuras de control

- **Función:** evaluar_ciclo_for(codigo_alumno: str) -> str
 - **Propósito Pedagógico:** Enseña la estructura del ciclo iterativo for, controlando que no se confunda la sintaxis con la de otros lenguajes como Java (validar que se use la función range() y la palabra clave in).
 - **Justificación:** Recibe el argumento de texto enviado en el comando de Discord, busca mediante expresiones regulares (re) si contiene la estructura correcta y devuelve una retroalimentación detallada o una alerta si detecta errores.
-
- **Función:** evaluar_ciclo_while(codigo_alumno: str) -> str
 - **Propósito Pedagógico:** Enseña la estructura del ciclo while y la importancia de la condición de paro, para que el alumno entienda por qué un ciclo mal planteado puede volverse infinito.
 - **Justificación:** Recibe el código propuesto por el alumno como texto. Mediante expresiones regulares verifica la presencia de la palabra clave while y de una condición booleana, y retorna un str con la retroalimentación correspondiente.
-
- **Función:** evaluar_condicional(codigo_alumno: str) -> str
 - **Propósito Pedagógico:** Enseña la estructura if-elif-else y el uso correcto de la indentación y los dos puntos, que suelen ser los errores más comunes de quien empieza a programar en Python.
 - **Justificación:** Recibe el texto del comando con el fragmento de código a evaluar. Retorna un str señalando si la estructura condicional está bien planteada o describiendo el error específico encontrado.

Estructuras de datos

- **Función:** `agregar_tarea(lista_tareas: list, descripcion: str) -> str`
 - **Propósito Pedagógico:** Enseña la manipulación de listas como estructura mutable, usando el método `append()` para agregar un elemento al final de la colección.
 - **Justificación:** Recibe la lista compartida de tareas y la descripción de texto a agregar. Retorna un str confirmando que la tarea fue añadida o señalando un error si la descripción está vacía.
-
- **Función:** `listar_tareas(lista_tareas: list) -> str`
 - **Propósito Pedagógico:** Enseña a recorrer una lista con `enumerate()` para mostrar su contenido numerado, reforzando el concepto de índice dentro de una colección.
 - **Justificación:** Recibe la lista de tareas. Retorna un str con cada tarea numerada o un mensaje indicando que la lista está vacía.
-
- **Función:** `eliminar_tarea(lista_tareas: list, indice_str: str) -> str`
 - **Propósito Pedagógico:** Enseña la eliminación de un elemento de una lista por posición con `pop()`, y la importancia de validar que el índice exista antes de operar sobre la colección.
 - **Justificación:** Recibe la lista de tareas y el índice como texto. Retorna un str confirmando la eliminación o un mensaje de error si el índice no es numérico o está fuera de rango.

Manejo de excepciones

- **Función:** `ejecutar_suma(argumento: str) -> str`
- **Propósito Pedagógico:** Enseña el flujo defensivo `try-except` aplicado a la conversión de tipos (de str a float), mostrando por qué Python no puede sumar texto como si fueran números.
- **Justificación:** Recibe el argumento de texto del comando con los dos números a sumar. Dentro de un bloque `try` intenta convertir y sumar los valores; el `except` captura cualquier fallo y retorna un str con el formato de uso correcto.

- **Función:** evaluar_excepcion(codigo_alumno: str) -> str
- **Propósito Pedagógico:** Enseña el uso de los bloques try-except y de la palabra clave raise, comparando el código que propone el alumno contra un manejo de excepciones correcto.
- **Justificación:** Recibe el código propuesto por el alumno como texto. Retorna un str indicando si el manejo de la excepción es adecuado o describiendo qué parte del bloque try-except falta o está mal estructurada.

Arquitectura del Agente

Las siguientes funciones no enseñan un tema de Python directamente al usuario, sino que conforman la arquitectura que permite que el bot funcione como un Agente (percepción, decisión y ejecución desacopladas) y no como un chatbot que resuelve todo en un único bloque de código.

- **Función:** clasificar_intencion(texto_limpio: str) -> str
- **Propósito Pedagógico:** No enseña un tema de Python al usuario; es la pieza que le permite al agente decidir qué hacer en lugar de seguir una cadena de if-elif-else fija, que es justo el comportamiento de chatbot que se busca evitar.
- **Justificación:** Recibe el texto ya limpio del mensaje del usuario. Analiza palabras clave para determinar la intención (saludo, solicitud de menú, nombre de un tema, petición de cuestionario, etc.) y retorna un str con el identificador de esa intención.
- **Función:** TOOLKIT: dict[str, dict] (catálogo de herramientas)
- **Propósito Pedagógico:** Enseña, a nivel de diseño del equipo, el concepto de un catálogo de herramientas: en vez de que el bot llame directo a cada función, primero la registra con su descripción para que el agente decida cuál invocar.
- **Justificación:** No es una función con argumentos sino una estructura de datos (diccionario) que recibe, por cada herramienta, su nombre como llave y un valor con la función a ejecutar y su descripción. El agente lo consulta para resolver una intención y retorna el resultado de la función encontrada.

- **Función:** generar_cuestionario(tema: str) -> str
 - **Propósito Pedagógico:** Cierra el ciclo de aprendizaje: después de que el agente explica un tema, el usuario puede pedir que lo evalúe, reforzando el contenido de forma activa en lugar de solo lectura pasiva.
 - **Justificación:** Recibe el tema sobre el que se quiere generar el cuestionario, tomado del mismo mensaje del usuario o de la memoria de estado si no se especifica. Retorna un str con preguntas de opción múltiple más una pregunta abierta, tomadas de un banco interno organizado por tema.
-
- **Función:** actualizar_memoria_usuario(id_usuario: str, clave: str, valor) -> None
 - **Propósito Pedagógico:** Enseña, a nivel de diseño del equipo, la diferencia entre un programa stateless y uno que mantiene memoria de estado, requisito para que el agente no resuelva cada mensaje de forma aislada.
 - **Justificación:** Recibe el identificador de Discord del usuario, la clave a actualizar (por ejemplo perfil o ultimo_tema) y el valor correspondiente. No retorna nada; actualiza el diccionario anidado donde se guarda el estado de cada usuario.